

Contents

1	Diseño de Algoritmos - TP0	1
1.1	Parte 1: Análisis de complejidad	1
2	Parte 2: Estructuras de datos y operaciones	2
2.1	Ejercicio 3:	2
2.2	Ejercicio 4:	2
2.3	Ejercicio 5:	2
3	Ejercicio 5:	3
4	Parte 3: Diagnóstico conceptual	3
5	Parte 4 (opcional): Problema abierto	3

1 Diseño de Algoritmos - TP0

Este trabajo práctico tiene como objetivo repasar conceptos fundamentales sobre eficiencia, estructuras de datos y complejidad, así como diagnosticar el nivel de conocimientos previos de los estudiantes.

1.1 Parte 1: Análisis de complejidad

Analizar la complejidad temporal de los siguientes fragmentos de código. Justificar utilizando reglas de suma y producto.

Ejercicio 1:

```
1.for (i=1; i<n; i++) { // O(n)
2.for (j=1; j<=n; j++) { //O(n)
3.if (a[i] < a[j]) //O(max(if,else) + 1 )
4.resp = true; //O(1)
5.else
6.resp = false; //O(1)
7.k = i + j; //O(1)
} }
```

El orden total es $O(n^2)$

Ejercicio 2:

```
i = 1; //O(1)
while (i <= n) { //O(log n)
if (a[i] >= a[n]) a[n] = a[i]; //O(1)
i = i * 2;
```

}
El orden total es $O(\log n)$

2 Parte 2: Estructuras de datos y operaciones

2.1 Ejercicio 3:

Determinar la complejidad (Big-O) de las siguientes operaciones y justificar la respuesta:

- a. Insertar un elemento en la posición k de una Lista. $//O(n)$
- b. Poner un elemento en una Cola. $//O(1)$
- c. Buscar un determinado elemento en un Árbol Binario.

En el peor caso considerando que el árbol no está ordenado, es de $O(n)$

En el mejor caso el orden sería $O(\log n)$

- d. Eliminar un elemento en un Árbol Binario de Búsqueda.

Peor caso: Teniendo en cuenta que el árbol esté desbalanceado, el peor caso será $O(n)$

Mejor caso: Sí el árbol está balanceado y ordenado, el mejor caso será $O(\log n)$

- e. Insertar un elemento en una tabla Hash. $//O(1)$

2.2 Ejercicio 4:

Analizar las clases de API Java para manejo de estructuras de datos y sus respectivos métodos. Tomar Queue y Hashmap como ejemplo.

2.3 Ejercicio 5:

Un juego de ajedrez se puede plantear como el siguiente problema de decisión:

Dado un posicionamiento legal de las piezas de ajedrez y la información sobre qué mueve, determinar si ese lado puede ganar. ¿Es decidible este problema de decisión? Explicar.

Este algoritmo no es decidible, ya que no es posible determinar si por un movimiento de pieza las piezas blancas ganarán. Las posibles jugadas de un juego de ajedrez son 10^{120} . Teóricamente sí es decidible pero aplicado a la realidad no. Si bien tiene una cota superior finita, es imposible alcanzar ese número en un algoritmo con la tecnología actual.

3 Ejercicio 5:

Un determinado problema puede ser resuelto por un algoritmo cuyo tiempo de ejecución está en $O(n \log^2 n)$. ¿Cuál de las siguientes afirmaciones es verdadera? Explicar.

El problema es manejable.

Es manejable porque su crecimiento si bien es inclinado, es más suave que x^2 .

El problema es intratable.

Imposible decirlo.

4 Parte 3: Diagnóstico conceptual

a) ¿Qué entendemos por eficiencia de un algoritmo?

La eficiencia de un algoritmo la determina el tiempo, los recursos que se utilizan para ejecutar a este y el espacio utilizado (memoria), todo esto considerando el peor caso.

b) ¿Qué estructuras de datos conocés y usás con mayor seguridad?

Lista, árbol, hashmap.

5 Parte 4 (opcional): Problema abierto

Dado un conjunto de números enteros positivos, diseñar un algoritmo (o describir en pasos) que determine si es posible dividir el conjunto en dos subconjuntos cuya suma sea igual. No es necesario programarlo, pero se valorará la claridad de la explicación y el enfoque algorítmico.

Algoritmo:

1. Sumar todos los números del conjunto grande.
- 2.